# HDOC Day 41

## Aim

To develop a menu-driven C program that reads two paragraphs and uses separate user-defined functions and pointers to find longest and shortest words, remove repeated spaces, convert text to title case, and display case-insensitive word frequencies.

## Algorithm

1. Start and read two paragraphs using fgets() in readParagraph().

2. Display a menu containing options 1 to 6 and read the user choice.

3. For option 1, read or re-enter both paragraphs.

4. For option 2, tokenize a copy of each paragraph and find the longest and shortest words.

5. For option 3, scan each paragraph and replace consecutive whitespace with one space.

6. For option 4, convert the first alphabetic character of each word to uppercase and remaining characters to lowercase.

7. For option 5, tokenize lowercase copies of the paragraphs, count matching words, and print frequencies.

8. Repeat until option 6 is selected, then stop.

## C Program

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stdlib.h>

#define MAX 1000
#define WORDS 500

void readParagraph(char paragraph[], size_t size) {
    if (fgets(paragraph, (int)size, stdin) == NULL) {
        paragraph[0] = '\0';
        return;
    }
    paragraph[strcspn(paragraph, "\n")] = '\0';
}

void findLongestAndShortestWords(const char paragraph[],
                                 char longestWord[],
                                 char shortestWord[]) {
    char copy[MAX];
    char *word;
    size_t longest = 0, shortest = MAX + 1;
    strcpy(copy, paragraph);
    longestWord[0] = shortestWord[0] = '\0';
    word = strtok(copy, " \t\r\n.,!?;:\"()[]{}");
    while (word != NULL) {
        size_t length = strlen(word);
        if (length > longest) {
            longest = length;
            strcpy(longestWord, word);
        }
        if (length < shortest) {
            shortest = length;
            strcpy(shortestWord, word);
        }
        word = strtok(NULL, " \t\r\n.,!?;:\"()[]{}");
    }
```

```
}

void removeRepeatedSpaces(const char paragraph[],
                          char modifiedParagraph[]) {
    int i = 0, j = 0, previousSpace = 0;
    while (paragraph[i] != '\0') {
        unsigned char ch = (unsigned char)paragraph[i++];
        if (isspace(ch)) {
            if (!previousSpace && j > 0) {
                modifiedParagraph[j++] = ' ';
            }
            previousSpace = 1;
        } else {
            modifiedParagraph[j++] = (char)ch;
            previousSpace = 0;
        }
    }
    if (j > 0 && modifiedParagraph[j - 1] == ' ') j--;
    modifiedParagraph[j] = '\0';
}

void convertToTitleCase(const char paragraph[],
                        char titleCaseParagraph[]) {
    int i, newWord = 1;
    for (i = 0; paragraph[i] != '\0'; i++) {
        unsigned char ch = (unsigned char)paragraph[i];
        if (isalpha(ch)) {
            titleCaseParagraph[i] = (char)(newWord ?
                                    toupper(ch) : tolower(ch));
            newWord = 0;
        } else {
            titleCaseParagraph[i] = (char)ch;
            if (isspace(ch) || ispunct(ch)) newWord = 1;
        }
    }
    titleCaseParagraph[i] = '\0';
}

void findWordFrequencies(const char paragraph[]) {
    char copy[MAX], words[WORDS][MAX];
    int counts[WORDS] = {0};
    int total = 0, i, found;
    char *word;
    strcpy(copy, paragraph);
    for (i = 0; copy[i] != '\0'; i++)
        copy[i] = (char)tolower((unsigned char)copy[i]);
    word = strtok(copy, " \t\r\n.,!?;:\"()[]{}");
    while (word != NULL) {
        found = -1;
        for (i = 0; i < total; i++) {
            if (strcmp(words[i], word) == 0) {
                found = i;
                break;
            }
        }
        if (found >= 0) counts[found]++;
        else if (total < WORDS) {
            strcpy(words[total], word);
            counts[total++] = 1;
        }
        word = strtok(NULL, " \t\r\n.,!?;:\"()[]{}");
    }
    if (total == 0) printf("No words found.\n");
    for (i = 0; i < total; i++)
        printf("%s : %d\n", words[i], counts[i]);
}

int main(void) {
    char paragraphs[2][MAX] = {{0}};
    char longest[MAX], shortest[MAX], modified[MAX];
    char input[32];
    int choice, i;
    printf("Enter paragraph 1: ");
    readParagraph(paragraphs[0], MAX);
```

```
    printf("Enter paragraph 2: ");
    readParagraph(paragraphs[1], MAX);

    do {
        printf("\n--- MENU ---\n");
        printf("1. Read two paragraphs\n");
        printf("2. Longest and shortest words\n");
        printf("3. Remove repeated spaces\n");
        printf("4. Convert to title case\n");
        printf("5. Word frequencies\n");
        printf("6. Exit\nEnter choice: ");
        if (fgets(input, sizeof(input), stdin) == NULL) break;
        choice = atoi(input);
        if (choice == 1) {
            for (i = 0; i < 2; i++) {
                printf("Enter paragraph %d: ", i + 1);
                readParagraph(paragraphs[i], MAX);
            }
        } else if (choice >= 2 && choice <= 5) {
            for (i = 0; i < 2; i++) {
                printf("\nParagraph %d:\n", i + 1);
                if (choice == 2) {
                    findLongestAndShortestWords(paragraphs[i],
                                                longest, shortest);
                    printf("Longest word: %s\n", longest);
                    printf("Shortest word: %s\n", shortest);
                } else if (choice == 3) {
                    removeRepeatedSpaces(paragraphs[i], modified);
                    printf("Modified: %s\n", modified);
                } else if (choice == 4) {
                    convertToTitleCase(paragraphs[i], modified);
                    printf("Title case: %s\n", modified);
                } else {
                    findWordFrequencies(paragraphs[i]);
                }
            }
        } else if (choice == 6) {
            printf("Program exited.\n");
        } else {
            printf("Invalid choice. Try again.\n");
        }
    } while (choice != 6);
    return 0;
}
```

## Sample Output

```
Enter paragraph 1: Hello   world hello
Enter paragraph 2: C   programming is FUN

--- MENU ---
Enter choice: 2
Paragraph 1:
Longest word: Hello
Shortest word: Hello
Paragraph 2:
Longest word: programming
Shortest word: C

Enter choice: 3
Paragraph 1:
Modified: Hello world hello
Paragraph 2:
Modified: C programming is FUN

Enter choice: 4
Paragraph 1:
Title case: Hello   World Hello
Paragraph 2:
Title case: C   Programming Is Fun

Enter choice: 5
Paragraph 1:
```

```
hello : 2
world : 1
Paragraph 2:
c : 1
programming : 1
is : 1
fun : 1

Enter choice: 6
Program exited.
```

## Result

The menu-driven C program was successfully developed to perform all five paragraph operations using user-defined functions, string handling, and pointer-based output parameters.